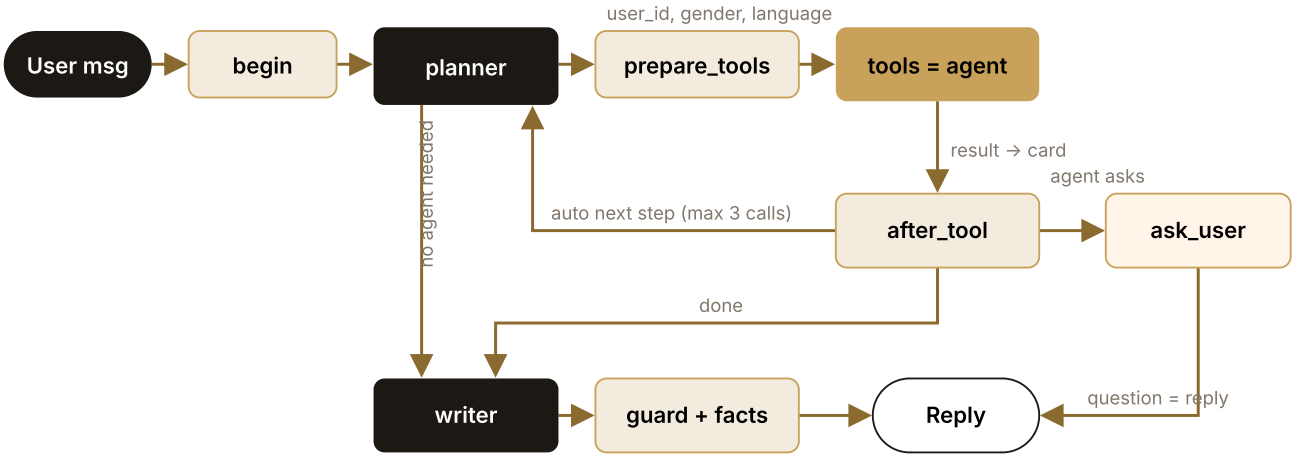
DressMe Orchestrator

How the assistant chat works — BO8 · `backend/orchestrator/` · LangGraph

> **In one sentence:** one chat, many agents. The orchestrator reads the message, picks the right agent (Smart Buy, Stylist, Local shops…), calls it as a tool, then writes a short answer from that agent's real facts. Rules route first (fast, exact, works with typos / French / Tounsi); the LLM is only the fallback.

1. The graph (one turn)



Node	What it does
`begin`	Starts the turn. Drops an old pending question if the user writes a new clear request.
`planner`	Decides, in this order: ① small talk (hello, thanks, who are you) → fixed answer · ② off-topic (maths, football, code) → polite decline · ③ "what am I missing?" → wardrobe gaps from M1 · ④ clear intent but missing detail → ask ("Which piece is it?") · ⑤ rule match → call an agent · ⑥ answer to an agent's question (yes / no) · ⑦ else the LLM chooses a tool.
`prepare_tools`	Adds what the user didn't say: `user_id`, gender from the account (for local shops), `language=en`.
`tools`	Runs the agent. Any error becomes a friendly sentence (`handle_tool_errors`), never a 503 or a traceback.
`after_tool`	Saves the result as a card for the app, the last agent, and the agent's question if it asked one.
`ask_user`	The agent needs info ("Do you need it for an event?") → that question is the reply.
`writer`	Writes the answer from **this turn's** facts with templates (BUY ✅ / THINK 🤔 / SKIP ✋, outfits, shops…). No template → LLM, then `guard` + `enforce_facts` (never contradict the verdict, never "JSON / function / I can't").

2. Understanding the message (before any LLM)

`understand.py` reads the sentence with rules, because an 8B local LLM is not reliable for routing.

- **Normalising:** lower case, `320dt / 320 dinars → 320 tnd`.
- **The piece:** type words (costume → blazer, pull → sweater, sbadri → sneakers), colour words (noir, k7al, bleu marine...), price.
- **The intent**, in priority order: gaps → localshop → smartbuy → challenges → wardrobe → tryon → laundry → stylist → profile → taste.
- **Follow-ups** with memory: "and in black?", "cheaper?" (×0.7), "in Sousse", "for a wedding instead" reuse the last request.

```
"shoul i by a blue  costume with 320dt"
 → intent smartbuy · piece {type: blazer, colour: blue, price: 320}
 → smartbuy(probe_image="custom:blazer:blue", price_tnd=320)

"chnowa nelbes ghodwa lel khedma"  → stylist (outfit for tomorrow, work)
"what m i missing for a wedding"   → gaps (formal pieces only)
```

3. The agents it can call

Found automatically by `registry.py`: every `modules/mXX/tool.py` is a tool. A module whose card names another agent is a **skill** of that agent (not called directly).

Agent (tool)	Module / BO	Its skills
stylist	BO2 outfits	M2 recommend, M3 wear log, M4 calendar, M17 capsule
smartbuy	M6 · BO5	M13 partner links, M18 size & fit
localshop	M7 · BO6	M14 boutique / stylist mode
wardrobe	M1	—
profile	M5 style profile	M12 style evolution
taste	M19	—
tryon	M9 · BO9 avatar	—
share / challenges	M10 / M15 · BO7	—
trips	M11	M16 sustainability
talk	M20	—

M8 (LLM layer) is the orchestrator's own skill: the brain. Wardrobe gaps come from `shared/wardrobe_gaps.py` (10 essentials checked on the M1 wardrobe).

4. Memory and state

Key	Kept for
`session_id`	One conversation (LangGraph `MemorySaver`, thread = session).
`intent`	A pending "which piece?" — the next message completes it.

`memory`	Last request {agent, piece, text} — used by follow-ups.
`pending`	An agent's question + its `question_key`, so "yes" goes back to the right agent.
`attachment`	A photo, read by FashionCLIP (type + colour) → Smart Buy.

5. The LLM (brain)

- `shared/llm.py`: providers **groq** (recommended, `llama-3.3-70b-versatile`), openai, ollama, anthropic. Temperature 0.
- No key → falls back to local Ollama.
- Used only for: free fashion questions ("how do I wash wool?"), and choosing a tool when no rule matched.
- Its answer always passes `guard`: an empty, technical or "I can't answer" reply is replaced by a helpful one.

6. What the app receives

`POST /chat/message {user_id, session_id, message, image?}` returns:

```
{ "reply": "BUY ✅ — blue blazer, score 72/100. It completes 4 outfits.",
  "question": null,            // set when the agent asks something
  "cards": [ {...agent result...} ],   // verdict ring, outfits, shops in the app
  "next_actions": [...],       // buttons
  "last_agent": "smartbuy",
  "trace": [                   // the workflow chain shown under the answer
    {"step":"understand","title":"Orchestrator understood","detail":"intent: smartbuy ·
type: blazer..."},
    {"step":"route","agent":"smartbuy","title":"Asked Smart Buy (BO5)"},
    {"step":"agent","agent":"smartbuy","ok":true,"title":"Smart Buy (BO5)
answered","detail":"verdict: buy, score: 72"},
    {"step":"reply","title":"Reply written","detail":"from the agent's facts"} ] }
```

In the app: **Orchestrator → Smart Buy (BO5) → Orchestrator · answer** (tap for the details).

7. Safety nets (why it always answers)

- Small talk and off-topic answered without the LLM.
- Agent error → "I couldn't finish... send a photo / try again" instead of an error.
- Writer only uses this turn's result (no old answer reused).
- Max 3 agent calls per turn.
- Mojibake / doubled words cleaned (`clean()`).
- Tests: `orchestrator/tests/test_jury_scenarios.py` (routing in EN / FR / Tounsi, conversations, memory, gaps, workflow) — full backend suite 758 passed.

8. Files: where they are and what they do

Part	File (position)	Lines	Objective
Core	`backend/orchestrator/graph.py`	591	The LangGraph graph: nodes begin, planner, prepare_tools, tools, after_tool, ask_user, writer. Rule routing (route_by_rules), arguments per agent (args_for), friendly errors, enforce_facts, run_turn and the workflow trace.
Core	`backend/orchestrator/understand.py`	144	Reads the message before any LLM: normalising (320dt → 320 tnd), piece type / colour / price in EN, FR, Tounsi, the intent, and follow-ups ("and in black?", "cheaper?").
Core	`backend/orchestrator/replies.py`	216	Answers written from the agent's facts (BUY / THINK / SKIP, outfits, shops...), small talk, off-topic decline, guard (no technical reply), GENERAL_PROMPT, gaps reply, occasion gaps, clean().
Core	`backend/orchestrator/registry.py`	54	Finds the agents automatically: every modules/mXX/tool.py becomes a tool; a module whose card names another agent is a skill of it.
Core	`backend/orchestrator/state.py`	19	The state of one conversation: messages, intent, memory, pending, attachment, cards, last_agent.
Core	`backend/orchestrator/prompts.py`	15	SYSTEM_PROMPT (tool choice) and WRITER_PROMPT for the LLM.

Core	`backend/orchestrator/studio.py`	55	The graph for the local LangGraph Studio viewer (to show the graph live).
API	`backend/api/routes/chat.py`	124	POST /chat/message — the app's single entry. Reads the photo (FashionCLIP), runs the turn, never returns a 503; GET /chat/agents lists the agent cards; LLM status.
Shared	`backend/shared/llm.py`	27	One place to get the LLM: groq / openai / ollama / anthropic, temperature 0.
Shared	`backend/shared/wardrobe_gaps.py`	95	What the user's M1 wardrobe is missing (10 essentials). Used by the assistant, Smart Buy, M5 /gaps and BO1.
Shared	`backend/shared/user_wardrobe.py`	99	The signed-in user's real M1 wardrobe in the format the outfit models need.
Agents	`backend/modules/*/tool.py + identity.py`	—	Each agent exposed as a tool (tool.py) with its card (identity.py: name, role, skills, contract).
App	`mobile/lib/features/bo8_assistant/ui/bo8_assistant_screen.dart`	—	The chat screen: text, voice, photo, agent cards, the workflow chain Orchestrator → agent → answer.
App	`mobile/lib/features/bo8_assistant/ui/agent_cards.dart`	—	How each agent's result is drawn: verdict ring, outfits, shop products, pieces, challenges, styles.
Tests	`backend/orchestrator/tests/test_jury_scenarios.py`	247	Jury sentences (typos, FR, Tounsi), full conversations, memory, small talk, off-topic, gaps, workflow.
Tests	`backend/orchestrator/tests/test_graph.py · test_chat_api.py · test_cards.py · test_registry.py · test_wardrobe_gaps.py ·`	—	The loop, the HTTP route, agent cards,

	`test_user_wardrobe.py · test_auth.py`		gaps, the real wardrobe, sign-in.
Script	`backend/scripts/assistant_check.py`	83	Live rehearsal: runs 8 real conversations against the running backend and shows who answered.